

Probeklausur – Flottenmanagementsystem

Bearbeitungszeit: 2h

Typ: Einzelarbeit

1. Problemstellung

Ein Logistikunternehmen möchte seine Fahrzeugflotte digital verwalten. Dafür soll ein objektorientiertes Softwaresystem in C++ entwickelt werden.

Die Fahrzeugflotte besteht aus unterschiedlichen Fahrzeugtypen:

- PKW
- Elektrofahrzeuge

All diese Fahrzeuge besitzen gemeinsame Eigenschaften:

- eine eindeutige Fahrzeug-ID
- den Hersteller
- Verfügbarkeitsstatus (verfügbar oder nicht verfügbar)
- die benötigte Führerscheinklasse
- der momentan zugewiesene Fahrer

Ein PKW hat zusätzlich noch Informationen über seinen Verbrauch pro 100Km gespeichert. Bei Elektrofahrzeugen interessiert insbesondere die Batteriekapazität. Neu erstellte Fahrzeuge sollen zu Beginn verfügbar sein und keinen zugewiesenen Fahrer haben.

Zu jedem Objekt eines Fahrzeugtyps lassen sich mit der Funktion `printInfo()` alle Attribute in einer Tabelle mit zwei senkrecht ausgerichteten Spalten auf dem Terminal ausgeben.

Zwei Ausgabebeispiele für ein Objekt vom Typ "PKW" und "Elektrofahrzeug" sehen wie folgt aus:

Type	PKW
Brand	Volkswagen
Consumption	6.5
Available	Yes
Needed License	B
Assigned Driver	None

Type	Electric Car
Brand	Tesla
Battery Capacity	75 kWh
Available	No
Needed License	B
Assigned Driver	Michael Schumacher

Auch die Daten der angestellten Fahrer werden über die Software verwaltet. Ein Fahrer hat eine ID, einen Namen und eine Liste seiner Führerscheinklassen (z.B. B, BE, C1). Die ID soll nur einmalig gesetzt werden können und muss eindeutig über alle Fahrer hinweg sein. Da sich die Führerscheinklassen ändern können, soll ein passender Container aus der Standard-Bibliothek verwendet werden, der geordnet oder durchsucht werden kann. Es sollen Methoden vorhanden sein, um neue Führerscheinklassen für einen Fahrer anzulegen oder bestehende Klassen zu löschen.

Fahrer können maximal ein Fahrzeug ausleihen, aber nur wenn dieses Fahrzeug verfügbar ist, der Fahrer kein anderes Fahrzeug momentan ausgeliehen hat und er die passende Führerscheinklasse besitzt.

Weitere Requirements:

- Verwenden Sie mindestens eine abstrakte Klasse und ein statisches Attribut.
- Orientieren Sie sich am Prinzip der Datenkapselung und implementieren Sie die nötigen Getter & Setter Methoden.
- Stellen Sie sicher, dass Ihr Code wiederverwendbar und erweiterbar ist.

2. Abgabebumfang

Folgende Aufgaben müssen bearbeitet werden:

- a) Erstellen Sie ein UML-Klassendiagramm, das als Basis für ein geplantes Softwaresystem dient und die oben beschriebenen Anforderungen erfüllt.
Darzustellen sind:
 - Alle Attribute und Methoden
 - Sichtbarkeiten
 - Datentypen
 - Zusätzliche Eigenschaften bei konstanten, statischen, abgeleiteten, eindeutigen, virtuellen, usw. Methoden/Attributen/Parametern
 - Parameterrichtungen
 - Klassenbeziehungen mit Navigierbarkeit, Multiplizität und Beziehungsnamen

Hinweis: Um Zeit bei Ihrem Diagramm zu sparen, genügt es pro Klasse jeweils nur eine Getter- und Setter-Methode darzustellen.

- b) Implementieren Sie nun das Softwaresystem für die Verwaltung der oben beschriebenen Fahrzeugflotte in C++. Folgen Sie dabei der Coding Convention für C++ aus der Vorlesung.

Wichtig: Orientieren Sie sich an Ihrem zuvor erstellten Klassendiagramm. D.h. der Code sollte alle Spezifikationen Ihres Klassendiagramms erfüllen.

Erweitern Sie den Code auch um alle technischen Details, die typischerweise nicht in Klassendiagrammen abgebildet werden, aber aus der Aufgabenbeschreibung hervorgehen und für die korrekte Umsetzung der Software notwendig sind.

Hinweis 1: Hierbei müssen nun alle relevanten Getter- & Setter-Methoden implementiert werden.

Hinweis 2: Sie müssen KEINE Funktions-Header, wie in den Coding Conventions gefordert, dokumentieren.

- c) Erzeugen Sie für jeden Fahrzeugtyp ein Objekt sowie einen Fahrer in Ihrer main-Funktion. Speichern Sie alle Fahrzeuge in einen Vector-Container.
Speichern Sie alle Fahrer in einen Vector-Container.
Weißten Sie dem Fahrer ein Fahrzeug zu. Zum Schluss geben Sie in einer Schleife die Daten aller Fahrzeuge über einen Basisklassenzeiger aus.
- d) Das Programm muss erfolgreich/fehlerfrei auf Ihrer Prüfungsmaschine kompilierbar sein.
- e) Beantworten Sie die folgende Frage direkt in Ihrem Code:
Wie können Sie bei polymorphen Klassen sicherstellen, dass die korrekte virtuelle Funktion überschrieben wird?